

Practical 4: Degree of Separation Using BFS

Objective: To understand the "Degree of Separation" concept and implement an algorithm (Breadth-First Search) to calculate it between any two nodes in a graph, analyzing its real-world applications.

Theory:

1. Introduction: The Concept of Degree of Separation

The "Degree of Separation" is a measure of social distance between two individuals within a network. It represents the minimum number of connections (or "hops") required to link one individual to another. This concept is famously illustrated by the "Six Degrees of Separation" theory, which suggests that any two people on Earth are, on average, approximately six or fewer social connections away from each other. In essence, it quantifies how interconnected we are through social networks.

2. Graph Theory Representation

In the context of graph theory, the Degree of Separation problem translates to finding the shortest path between two vertices (nodes) in an unweighted graph.

- **Nodes (Vertices):** Represent individuals or entities within the network.
- **Edges (Connections):** Represent relationships or connections between individuals. An edge between two nodes indicates a direct connection (e.g., friendship, acquaintance).
- **Unweighted Graph:** In this context, all connections are considered equal; there's no concept of stronger or weaker ties. Each edge has the same "weight" (usually implicitly 1).
- **Shortest Path:** The path with the fewest number of edges connecting two nodes. The length of this shortest path is the Degree of Separation.

3. Algorithm: Breadth-First Search (BFS)

The Breadth-First Search (BFS) algorithm is the most appropriate choice for calculating the Degree of Separation in an unweighted graph. It systematically explores the graph layer by layer, starting from a source node. This ensures that the first time the algorithm reaches the target node, it has discovered the shortest possible path.

4. Applications of Degree of Separation

- **Social Networks:**
 - *Connection Recommendations:* Used by platforms like LinkedIn and Facebook to suggest new connections based on network proximity ("2nd degree connections," "People You May Know").
 - *Privacy Settings:* Influences how information is shared with connections of connections.
- **Network Routing:** Finding the optimal path for data packets to travel between two points in a computer network, minimizing the number of hops.
- **Recommendation Engines:** Suggesting new content, products, or connections based on the user's proximity to other users or items in a network.
- **Epidemiology:** Modeling the spread of diseases through social networks to identify key individuals or communities for targeted interventions and containment strategies.
- **Bioinformatics:** Analyzing protein-protein interaction networks to understand functional relationships and dependencies between different proteins.

Degree of Seperation

```
In [16]: from collections import deque
import pandas as pd

def build_graph(edge_dict):
    graph = {}
    for entry in edge_dict.values():
        a, b = entry['A'], entry['B']
        if a not in graph:
            graph[a] = []
        if b not in graph:
            graph[b] = []
        graph[a].append(b)
        graph[b].append(a)
    return graph

def degree_of_separation(graph, person1, person2):
    if person1 == person2:
        return 0
    visited = set()
    queue = deque([(person1, 0)])
    visited.add(person1)
    while queue:
        current, degree = queue.popleft()
        for neighbor in graph.get(current, []):
            if neighbor == person2:
                return degree + 1
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, degree + 1))
    return -1 # No connection found
```

```
In [17]: edge_dict = pd.read_excel('data.xlsx', index_col=0).to_dict(orient='index')
graph = build_graph(edge_dict)
graph
```

```
Out[17]: {'person1': ['person2'],
'person2': ['person1'],
'Alice': ['Bob', 'Eve'],
'Bob': ['Alice', 'Charlie'],
'Charlie': ['Bob', 'David', 'Hector'],
'David': ['Charlie'],
'Eve': ['Alice', 'Frank'],
'Frank': ['Eve', 'Grace'],
'Grace': ['Frank'],
'Hector': ['Charlie', 'Isabella'],
'Isabella': ['Hector', 'James'],
'James': ['Isabella']}
```

```
In [18]: degree_of_separation(graph, 'Alice', 'Charlie')
```

```
Out[18]: 2
```